

Python API

Additional information on use of Python in Cisco Packet Tracer can be found at the end of this page.

Program Structure and Events

Package = N/A

Function	Return Type	Description	Example
cleanUp()	N/A	If defined, this function is called once just before the program stops.	<pre>def cleanUp(): print("program is stopping.")</pre>
mouseEvent(pressed, x, y, firstPress)	N/A	If defined, this function is called when the user clicks and/or moves the mouse on the workspace icon of this device. - pressed - a boolean indicating whether the left mouse button is pressed down - x - the x coordinate (in pixels) of the mouse relative to the workspace icon's top left corner - y - the y coordinate (in pixels) of the mouse relative to the workspace icon's top left corner - firstPress - a boolean indicating whether the left mouse button is the first time being pressed down after the last call to this function; pressed is true when the mouse button is pressed and moved, but firstPress is true only when the mouse button is first pressed but not when moved.	<pre>def mouseEvent(pressed, x, y, firstPress): if firstPress: doSomething()</pre>
measurementSystemChangeEvent()	N/A	If defined, this function is called when the user changes the measurement system between Metric and Imperial in Preferences. Use Options.isUsingMetric() in the options package to get the current setting.	<pre>from options import * def measurementSystemChangeEvent(): METRIC = Options.isUsingMetric() unit = "C" if METRIC else "F" refresh()</pre>

Digital I/O

Package = gpio

Function	Return Type	Description	Example
pinMode(slot, mode)	N/A	Set a digital slot to INPUT or OUTPUT.	<pre>pinMode(1, OUT) pinMode(2, IN)</pre>
digitalRead(slot)	int	Reads from a digital slot, returns HIGH or LOW.	val = digitalRead(1)
digitalWrite(slot, value)	N/A	Writes to a digital slot with HIGH or LOW.	digitalWrite(1, HIGH)

Analog I/O

Package = gpio

Function	Return Type	Description	Example
analogRead(slot)	int	Reads from an analog slot, returns 0 to 1023.	val = analogRead(A1)
analogWrite(slot, value)	N/A	Writes a PWM wave to a digital slot, from 0 to 255.	analogWrite(A1, 128)

Custom I/O

Package = gpio

Function	Return Type	Description	Example
customRead(slot)	str	Reads from an custom slot, returns a String	val = customRead(1)

customWrite(slot, value) N/A Writes a string to a digital slot. You can use customRead directly from the other side customWrite(1, "hello")

Input Interrupts/Events Package = gpio

Function	Return Type	Description	Example
add_event_detect(slot, callback)	N/A	Registers a function to be called when the input of a slot changes. This works for analog, digital and custom inputs. Whenever the input changes, the callback is called. Only one function is registered per slot. Calling this a second time for the same slot will remove the first callback.	def detect(): input = analogRead(0) # do something add_event_detect(0, detect)
remove_event_detect(slot)	N/A	Unregisters the slot for input changes.	remove_event_detect(0)

Time Package = time

Function	Return Type	Description	Example
delay(tmsec)	N/A	Pauses the program for tmsec milliseconds. Value of 0 has the meaning of sleeping until the next execution cycle, as determined by the run-time.	delay(1700) same as sleep(1.7)
sleep(tsec)	N/A	Pauses the program for tsec seconds. Fractional seconds values for tsec are acceptable. Value of 0 has the meaning of sleeping until the next execution cycle, as determined by the run-time.	sleep(0.25) same as delay(250)
uptime()	float	Returns the time since the device was started in seconds.	print(uptime())
gmtime([secs])	struct_time	Converts a time in seconds since epoch to struct_time in UTC. If secs is not provided, it uses the current time as returned by time().	print(gmtime())
localtime([secs])	struct_time	Converts a time in seconds since epoch to struct_time in local time. If secs is not provided, it uses the current time as returned by time().	print(localtime())
asctime([time])	string	Converts a time in struct_time to a string in the form of 'Sun Jun 20 23:21:05 1993'. If time is not provided, it uses the current time as returned by localtime().	print(asctime())
ctime([secs])	string	Converts a time in seconds since epoch to a string in the form of 'Sun Jun 20 23:21:05 1993'. If secs is not provided, it uses the current time as returned by time().	print(ctime())
mktime(time)	int	Converts a time in struct_time to a time in seconds since epoch.	mktime(localtime())
timezone	int	The local (non-DST) timezone's offset in seconds west of UTC.	print(timezone)
altzone	int	The local (DST) timezone's offset in seconds west of UTC.	print(altzone)
tzname	tuple(string, string)	A tuple of two strings with the first being the local non-DST timezone name, and second being the local DST timezone name.	print(tzname)
daylight	int	Nonzero if in a DST timezone.	print(daylight)
accept2dayear	bool	Whether two digit year will be accepted.	print(accept2dayear)
strftime(format[, time])	string	Converts a time in struct_time to a string in the format. Format directives: %a - Locale's abbreviated weekday name. %A - Locale's full weekday name. %b - Locale's abbreviated month name. %B - Locale's full month name. %c - Locale's appropriate date and time representation. %d - Day of the month as a decimal number [01,31]. %H - Hour (24-hour clock) as a decimal number [00,23]. %I - Hour (12-hour clock) as a decimal number [01,12]. %j - Day of the year as a decimal number [001,366]. %m - Month as a decimal number [01,12]. %M - Minute as a decimal number [00,59]. %p - Locale's equivalent of either AM or PM. %S - Second as a decimal number [00,61]. %U - Week number of the year (Sunday as the first day of the week) as a decimal number [00,53]. All days in a new year preceding the first Sunday are considered to be in week 0. %w - Weekday as a decimal number [0(Sunday),6]. %W - Week number of the year (Monday as the first day of the week) as a decimal number [00,53]. All days in a new year preceding the first Monday are considered to be in week 0. %x - Locale's appropriate date representation. %X - Locale's appropriate time representation. %y - Year without century as a decimal number [00,99]. %Y - Year with century as a decimal number.	print(strftime("%a, %d %b %Y %H:%M:%S +0000", gmtime()))

%Z - Time zone name (no characters if no time zone exists).
%% - A literal '%' character.

strptime(string[,
format])

struct_time

Converts a time in string format to a time in struct_time. The format parameter uses the same directives as those used by strftime(); it defaults to "%a %b %d %H:%M:%S %Y" which matches the formatting returned by ctime().

print(strptime("30 Nov 00", "%d %b %y"))

struct_time class		
tm_year	int	year, for example, 1993
tm_mon	int	month of year, range [1, 12]
tm_mday	int	day of month, range [1, 31]
tm_hour	int	hours, range [0, 23]
tm_min	int	minutes, range [0, 59]
tm_sec	int	seconds, range [0, 61]
tm_wday	int	day of week, range [0, 6], Monday is 0
tm_yday	int	day of year, range [1, 366]
tm_isdst	int	1 if summer time is in effect, 0 if not, and -1 if unknown

Basic Networking

Package = networking

Function	Return Type	Description	Example
localIP()	str	Returns the local IP.	ip = localIP()
subnetMask()	str	Returns the subnet mask.	mask = subnetMask()
gatewayIP()	str	Returns the gateway IP.	gateway = gatewayIP()

HTTP Client

Package = http

Function	Return Type	Description	Example
HTTPClient()	HTTPClient	Creates a HTTP Client.	http = HTTPClient()
open(url)	N/A	Gets an URL.	http.open("http://www.cisco.com")
stop()	N/A	Stops the request.	http.stop()
onDone(callback)	N/A	Sets the callback for when the request is done.	def onHTTPOne(status, data): print(data) ... http.onDone(onHTTPOne)

HTTP Server (SBC only)

Package = http

Function	Return Type	Description	Example
route(path, callback);	N/A	Sets up a route for path and calls callback when it is requested. Routes also support wildcards using *.	def onRouteHello(url, response): response.send("hello") HTTPServer.route("/hello", onRouteHello) def onRouteAll(url, response): response.send("world") HTTPServer.route("/*", onRouteAll)
start(port)	bool	Starts listening on port.	HTTPServer.start(80)
stop()	N/A	Stops listening.	HTTPServer.stop()

Response class Passed into the HTTPServer route handler.

send(content)	N/A	Sends content back as response.	response.send("hello")
setContentType(type)	N/A	Sets the content type in the response.	response.setContentType("text/plain")
sendFile(filePath)	N/A	Sends a file back as response. The file path is in the device's file manager, not relative to the source code of the current project/script.	response.sendFile("/test.txt")
sendNotFound()	N/A	Sends a file not found as response.	response.sendNotFound()

Email**Package = email**

Function	Return Type	Description	Example
setup(email, server, username, password)	N/A	Sets up the email client to be used.	EmailClient.setup("user@cisco.com", "cisco.com", "username", "password")
send(address, subject, body)	N/A	Sends an email.	EmailClient.send("pt@cisco.com", "subject", "body")
receive()	N/A	Receives emails.	EmailClient.receive()
onSend(callback)	N/A	Sets the callback for when sending an email is done.	def onEmailSend(success): print(success) ... EmailClient.onSend(onEmailSend)
onReceive(callback)	N/A	Sets the callback for when emails are received.	def onEmailReceive(sender, subject, body): print(body) ... EmailClient.onReceive(onEmailReceive)

TCP**Package = tcp**

Function	Return Type	Description	Example
TCPClient()	TCPClient	Creates a TCP Client.	client = TCPClient()
connect(ip, port)	bool	Connects to ip and port.	client.connect("1.1.1.1", 2000)
connected()	bool	Returns true if connected	client.connected()
close()	N/A	Disconnects.	client.close()
state()	int	Returns the state of the connection.	client.state()
remoteIP()	str	Returns the remote IP.	client.remoteIP()
remotePort()	int	Returns the remote port.	client.remotePort()
localIP()	str	Returns the local IP	client.localIP()
localPort()	int	Returns the local port	client.localPort()
send(data)	N/A	Sends data to remote connection	client.send("hello")
sendWithPDUInfo(data, pduInfo)	bool	Sends data to remote connection with PDU info.	data = "hello" pduInfo = PDUInfo(0xffff00) pduInfo.setOutFormat("MyProtocol", "MyPDU", .
onReceive(callback)	N/A	Sets the callback for	def onTCPReceive(data): print(data)

		when data is received.	... client.onReceive(onTCPReceive)
onReceiveWithPDUInfo(callback)	N/A	Sets the callback for when data is received and includes the PDU info.	client = TCPClient()...def onTCPReceiveWithPDUInfo(data, pduInfo): client.onReceiveWithPDUInfo(onTCPReceiveWithPDUInfo)
onConnectionChange(callback)	N/A	Sets the callback for when the connection changes.	def onTCPConnectionChange(type): print(type) ... client.onConnectionChange(onTCPConnectionChange)
TCPServer()	TCPServer	Creates a TCP Server.	server = TCPServer()
listen(port)	N/A	Starts listening on port.	server.listen(2000)
stop()	N/A	Stops listening.	server.stop()
onNewClient(callback)	N/A	Sets the callback for when a new client comes in.	def onTCPNewClient(client): print(client) ... server.onNewClient(onTCPNewClient)

UDP		Package = udp	
Function	Return Type	Description	Example
UDPSocket()	UDPSocket	Creates an UDP Socket.	udp = UDPSocket()
begin(port)	bool	Starts listening on port.	udp.begin(2000)
stop()	N/A	Stops listening.	udp.stop()
send(ip, port, data)	bool	Sends data.	udp.send("1.1.1.1", 2000, "hello")
sendWithPDUInfo(ip, port, data, pduInfo)	bool	Sends data with PDU info.	data = "hello" pduInfo = PDUInfo(0xffff00) pduInfo.setOutFormat("MyProtocol", "MyPDU", {"type": "REPLY", "data": data}) pduInfo.addOutMessage("I am sending some data.") udp.sendWithPDUInfo("1.1.1.1", 2000, data, pduInfo)
onReceive(callback)	N/A	Sets the callback for when data is received.	def onUDPReceive(ip, port, data): print(data) ... udp.onReceive(onUDPReceive)
onReceiveWithPDUInfo(callback)	N/A	Sets the callback for when data is received and includes PDU info.	def onUDPReceiveWithPDUInfo(ip, port, data, pduInfo): print(data) pduInfo.addInMessage("I got some data.") pduInfo.setAccepted(). udp.onReceiveWithPDUInfo(onUDPReceiveWithPDUInfo)

File (SBC only)		Package = file, os, os.path	
Function	Return Type	Description	Example
bool exists(path)	bool	Returns whether a file exists in the file system.	exists("/file.txt")
File open(path, mode)	File	Opens a file for reading or writing.	file = open("/file.txt", "r")

bool remove(path)	bool	Removes a file.	remove("/file.txt")
bool mkdir(path)	bool	Creates a directory including all intermediate directories.	mkdir("/dir1/dir2")
bool rmdir(path)	bool	Removes a directory.	rmdir("/dir1/dir2")
listdir(path)	list	Lists all files in a directory.	files = listdir("/dir1")
exists(path)	bool	Test if path exists	val = exists("/file")
isfile(path)	bool	Test if path is a file	val = exists("/file")
isdir(path)	bool	Test if path is a directory	val = exists("/file")
tell()	int	Return read/write pointer of the file	file.tell()
close()	N/A	Closes the file.	file.close()
seek(position)	bool	Seeks to position in file.	file.seek(0)
readline()	str	Reads a line of string or to the end of file.	val = file.readline()
read(length)	str	Reads the length of byte into string	val = file.read()
write(val)	N/A	Writes as binary.	file.write(val)

USB**Package = usb**

Function	Return Type	Description	Example
USB(usbNum, speed)	USB	The variable for USB port.	usb = USB(0, 9600);
close()	N/A	Ends communication.	usb.close();
inWaiting()	int	# of bytes available for reading in buffer.	bytes = usb.inWaiting();
readLine()	string	Reads a line of string or to the end of stream.	val = usb.readLine();
read()	string	Reads the first character and removes from buffer.	val = usb.read();
peek()	string	Reads one character without removing from buffer.	val = usb.peek();
write(val)	int	Prints to USB, returns the number of bytes written.	val = usb.write(val);
PTmata(usbNum, speed)	PTmata	The variable for PTmata communication over USB port.	ptmata = PTmata(0, 9600)
close()	N/A	Ends communication.	ptmata.close()
pinMode(slot, mode)	N/A	Set a digital slot on other side to INPUT or OUTPUT.	ptmata.pinMode(1, OUT) ptmata.pinMode(2, IN)
digitalRead(slot)	int	Reads from a digital slot on other side, returns HIGH or LOW.	val = ptmata.digitalRead(1)
digitalWrite(slot, value)	N/A	Writes to a digital slot on other side with HIGH or LOW.	ptmata.digitalWrite(1, HIGH)
analogRead(slot)	int	Reads from an analog slot on other side, returns 0 to 1023.	val = ptmata.analogRead(A1);
analogWrite(slot, value)	N/A	Writes a PWM wave to a digital slot on other side, from 0 to 255.	ptmata.analogWrite(A1, 128);
customRead(slot)	string	Reads from an custom slot on other side, returns a String	val = ptmata.customRead(1);
customWrite(slot, value)	N/A	Writes a string to a digital slot on other side. You can use customRead directly from the other side	ptmata.customWrite(1, "hello");
inWaiting()	int	# of bytes available for reading in buffer.	bytes = ptmata.inWaiting();
processInput()	N/A	Reads from buffer and processes inputs for commands and reports of states.	def loop() : while ptmata.inWaiting(): ptmata.processInput() ptmata.readAndReportData()

readAndReportData() N/A

Reads this side's slot values and report to other side if they are changed.

ptmata.readAndReportData();

IoE Client (SBC only)**Package = ioeclient**

Function	Return Type	Description	Example
IoEClient.setup(api)	N/A	Sets up the API for remote monitor and control from IoE server. The api is an object with the following properties describing the device: - type - a string for the type of this device - states - an Array of objects with the following properties describing the state: - name - a string for this state - type - a string for the type of this state; can be "bool", "number", "options", "string" - options (required if type is "options") - an object that maps values to names - unit (optional if type is "number") - the default or Metric unit label; the value of a number state sent to the IoE Server should be in this unit - imperialUnit (optional if type is "number") - the Imperial System unit label - toImperialConversion (optional if type is "number") - a string to be evaluated to convert the default value to Imperial unit where x is the default value - toMetricConversion (optional if type is "number") - a string to be evaluated to convert the value in Imperial unit to the default or Metric unit, where x is the Imperial value - decimalDigits (optional if type is "number") - the number of decimal digits to round to on IoE Server pages; default is to not round - controllable - a boolean indicating whether it is remotely controllable - minValue (required if type is "number" and controllable is true) - the minimum value to allow the IoE Server to set in default or Metric unit - maxValue (required if type is "number" and controllable is true) - the maximum value to allow the IoE Server to set in default or Metric unit For measurement systems other than Metric and Imperial, use only the "unit" property. That means if you want a device to show more than Metric and Imperial, you need to create another device for other measurement systems.	<pre>IoEClient.setup({ "type": "Door", "states": [{ "name": "Open", "type": "bool" }, { "name": "Lock", "type": "options", "options": { "0": "Unlock", "1": "Lock" } }, { "controllable": True } }]); IoEClient.setup({ "type": "Thermostat", "states": [{ "name": "Status", "type": "options", "options": { "0": "Off", "1": "Cooling", "2": "Heating", "3": "Auto" } }, { "controllable": True }, { "name": "Temperature", "type": "number", "unit": "&deg;C", "imperialUnit": "&deg;F", "toImperialConversion": "x*1.8+32", "toMetricConversion": "(x-32)/1.8", "decimalDigits": 1 }, { "name": "Auto Cool Temperature", "type": "number", "unit": "&deg;C", "imperialUnit": "&deg;F", "toImperialConversion": "x*1.8+32", "toMetricConversion": "(x-32)/1.8", "decimalDigits": 1, "controllable": True, "minValue": 10, "maxValue": 100 }, { "name": "Auto Heat Temperature", "type": "number", "unit": "&deg;C", "imperialUnit": "&deg;F", "toImperialConversion": "x*1.8+32", "toMetricConversion": "(x-32)/1.8", "decimalDigits": 1, "controllable": True, "minValue": -100, "maxValue": 20 } }]);</pre>
IoEClient.reportStates(states)	N/A	Reports the states of this device to the IoE server. The argument can be a string representing all states of this device. Each state is separated by a comma. The argument can also be an array representing all	<pre>IoEClient.reportStates("0,1") IoEClient.reportStates([0, 1, "str"])</pre>

states.

The number of states must match the length of the states property in setup().

IoEClient.onInputReceive(callback)	N/A	Sets the callback for processing inputs received from IoE server. The argument to the callback is a string containing all states of this device. This is called with all states info. onStateSet is called with only the state that was changed.	<pre>def onInputReceiveDone(input): print(input) ... IoEClient.onInputReceive(onInputReceiveDone)</pre>
IoEClient.onStateSet(callback)	N/A	Sets the callback for processing inputs received from IoE server. The arguments to the callback are state name and state value. This is called with only the state that was changed. onInputReceive is called with all states info.	<pre>def onStateSet(stateName, value): print(stateName + ": " + value) ... IoEClient.onStateSet(onStateSet)</pre>

Physical

Package = physical

Function	Return Type	Description	Example
move(x,y)	N/A	Move thing to position x and y in screen coordinates. The parameters expect x and y are ints. Casting may be required.	move(200,200)
moveBy(x,y)	N/A	Increment position of thing by x and y in screen coordinates. The parameters expect x and y are ints. Casting may be required.	moveBy(1,0)
moveItemInWorkspace(name, x, y)	bool	Moves the item defined by name to x and y in screen coordinates in the active workspace	moveItemInWorkspace("building", 300,300)
getX()	float	Gets the x position of thing in screen coordinates.	x = getX()
getY()	float	Gets the y position of thing in screen coordinates.	y = getY()
devicesAt(x, y, width, height)	list	Gets a list of devices at position x and y with a boundary of width and height. The parameters expect x and y are ints. Casting may be required.	devices = devicesAt(10,10,100,100)
devicesIncludingClustersAt(x, y, width, height)	Array of string	Gets a list of devices at position x and y with a boundary of width and height including clusters	devicesIncludingClustersAt(10, 10, 100, 100)
getName()	str	Gets the name of the thing	devName = getName()
getDeviceProperty(deviceName, property)	str	Gets the property of a device with the specified property	prop = getDeviceProperty("Car", "material")
setDeviceProperty(deviceName, property, value)	N/A	Set property for device	prop= setDeviceProperty("Car", "material", "metal")
setComponentOpacity(componentName, value)	N/A	Set the opacity of a component in the thing. The value is from 0 to 1, where 1 is opaque.	setComponentOpacity("light", 0.5)
setComponentRotation(componentName, value)	N/A	Sets the component rotation in degrees	setComponentRotation("hourHand", 90)

setThingRotation(value)	N/A	Sets the entire thing rotation in degrees	setThingRotation(180)
getSerialNumber()	str	Gets the serial number of the thing	serialNo = getSerialNumber()
setCustomText(x,y,width,height,text)	N/A	Write some text on the Thing viewable on the workspace.	setCustomText(0,0,100,100,"Device is On")
fillColor(componentName, red, green, blue)	N/A	Fill the component with the specified RGB values. The component original image will have to be transparent for the color to show up.	fillColor("led", 0,0,255)
addSound(soundID, soundPath)	N/A	Adds a sound to the device so it can be used. Sound is referenced by the ID for later use. PT sound folder is: "/../Sounds/"	addSound('sound1', '../Sounds/buzzLow.wav');
playSound(soundID, playLength)	N/A	Plays a sound that has been added to the device. soundID references the ID the sound was added with, playLength is how many times the sound should run. -1 will make the sound loop forever.	playSound('sound1', 2);
stopSound(soundID)	N/A	Stops a sound. soundID references the ID the sound played.	stopSound('sound1');
destroySounds()	N/A	Stops any sounds playing in the devices and removes them. They can't be played again unless re-added.	destroySounds();
setParentGraphicFromComponent(componentName, index)	N/A	Sets the parent container of both physical and logical view to a graphic from component.	setParentGraphicFromComponent("name", 0)
setLogicalParentGraphicFromComponent(componentName, index)	N/A	Sets the parent container of logical view to a graphic from component.	setLogicalParentGraphicFromComponent("name", 0)
setPhysicalParentGraphicFromComponent(componentName, index)	N/A	Sets the parent container of physical view to a graphic from component.	setPhysicalParentGraphicFromComponent("name", 0)
setLogicalBackgroundPath(path)	N/A	Sets the logical view background to an image at path.	setLogicalBackgroundPath("path")
getAttributeOfDeviceAtSlot(attribute, slot)	float	Returns the attribute value of the device connected at the specified slot.	value = getAttributeOfDeviceAtSlot("name", 0)
getAttributeOfDevice(attribute)	float	Returns the attribute value of this device.	value = getAttributeOfDevice("name")
getSlotsCount()	int	Returns the number of slots this device has.	slots = getSlotsCount()

Environment

Package = environment (from environment import *) or (from environment import Environment)

Function	Return Type	Description	Example
get(environmentID)	float	Gets the value of the environment by its ID. You can get the ID by placing your mouse over the environment name in the Environment GUI. If the environment does not exist, it will return -1.	Environment.get("Ambient Temperature")
setGlobalProperty(propertyName, value)	N/A	Sets a global property with a value. Both are strings.	Environment.setGlobalProperty("CLOCK", "12:00:00 pm")
getGlobalProperty(propertyName)	string	Returns the global property value.	Environment.getGlobalProperty("CLOCK")
hasGlobalProperty(propertyName)	boolean	Returns true if the property name exists, otherwise false.	Environment.hasGlobalProperty("CLOCK")
setContribution(environmentID, rate, limit, bCumulative)		Set the things contribution to an environment based on it's environment ID. You do not need to worry about how much time has passed and you only need to call this function when you need different parameters. rate: the rate to add or subtract from the total environment value in value/second. Value should be in metric. limit: the maximum or minimum this thing is allowed to contribute. The limit should be in metric. bCumulative: add up contributed values over time. For environments like light sources that disappear when off, bCumulative should be set to false.	// increase the Ambient Temperature at 0.05C/second until 100C. Environment.setContribution("Ambient Temperature", 0.05, 100, true)
removeCumulativeContribution(environmentID)		Remove the overall cumulative contribution from the Thing. In most cases, you do not need to do this. Rather, you should set up the container to use transference or other things to remove accumulated contributions.	Environment.removeCumulativeContribution("Ambient Temperature")
setTransferenceMultiplier(environmentID, multiplier)		Increase or decrease the current transference value by multiplier. For example, if you open the door to the house, you may want to speed up the Ambient Temperature convergence with the outside by increasing the container's transference by the multiplier.	Environment.setTransferenceMultiplier("Ambient Temperature", 2)
getTotalContributions(environmentID)	float	Returns the total value of contributions by all things	Environment.getTotalContributions("Wind Speed")
getCumulativeContribution(environmentID)		Returns the cumulativeContribution for just the thing that is calling the function	Environment.getCumulativeContribution("Wind Speed")
getMetricValue(environmentID)	float	Returns the metric value of the environmentID regardless of user preference	Environment.getMetricValue("Ambient Temperature")
getValueWithUnit(environmentID)	string	Returns the value in metric or imperial based on the user preference and also append the unit	Environment.getValueWithUnit("Ambient Temperature")
getUnit(environmentID)	string	Returns the unit for the environmentID. The unit can be metric or imperial based on the user preferences	Environment.getUnit("Ambient Temperature")
getVolume()	float	Returns the volume size of the container in meters^3 that caller of the function is in	Environment.getVolume()
getTimeInSeconds()	int	Returns the current time	Environment.getTimeInSeconds();

getElapsedTime(lastTime)	int	Returns the time passed since the lastTime value	var time = Environment.getTimeInSeconds(); delay(1000); Environment.getElapsedTime(time);
--------------------------	-----	--	---

Real HTTP Server (External Network Access)

Package = realhttp

Function	Return Type	Description	Example
RealHTTPServer()	RealHTTPServer	Creates a Real HTTP Server.	http = RealHTTPServer()
start(port)	N/A	Start server listening on <i>port</i> .	http.start(8765)
stop()	N/A	Stop server listening.	http.stop()
isListening()	boolean	Returns the listening status of the server.	if http.isListening() : ...
route(url-pattern, request-types, callback, context=None)	N/A	Adds <i>callback</i> to be run for all <i>request-types</i> matching <i>url-pattern</i>. <i>callback: function (context, request, reply);</i> <i>request: RealHttpServerRequest</i> <i>response: RealHttpServerResponse</i> <i>url-pattern: string</i> <i>request-types: array of strings</i> ("GET", "POST", "PUT", "UPDATE", "DELETE") <i>context: object.</i> If <i>None</i> the <i>server</i> object will be passed in as context.	<pre>def on_get_files(context, request, reply): ... def on_posts(context, request, reply): ... def on_everything(context, request, reply): ... http.route("/files/*", ["GET"], on_get_files) http.route("**", ["POST"], on_posts) http.route("**", [], on_everything)</pre>
websocket(url-pattern, callback)	N/A	Adds <i>callback</i> to be run for incoming websocket connections matching <i>url-pattern</i>. <i>callback: function (client);</i> <i>client: RealWSClient</i> <i>url-pattern: string</i>	<pre>def callback(client): pass server.websocket("/ws", callback)</pre>
RealHTTPServerRequest		Request object passed to routing callbacks.	
headers()	object	Returns object containing request headers.	headers = request.headers() ... headers["Content-Type"] ...
method()	string	Returns request's method ("GET", etc.)	if request.method() == "POST": ...
body()	string	Returns the body of the request.	request.body();
url()	string	Returns requested URL.	request.url();
ip()	string	Returns IP address of the request origin.	if request.ip() == "1.1.1.1": ...
RealHTTPServerResponse		Response object passed to routing callbacks.	
setStatus(code)	N/A	Sets response status to <i>code</i> .	response.setCode(200) # for OK response
		code: number	

setHeaders(headers)	N/A	Sets response headers in bulk. Object properties define header names and values.	response.setHeaders({ "Authorization": "basic", "Cookie": "Id=1234; Group=users" })
		headers: object	
addHeader(name, value)	N/A	Sets header <i>name</i> to <i>value</i> .	response.addHeader("Cookie", "Group=inner")
		name: string value: string	
appendContent(content)	N/A	Appends <i>content</i> to response body.	response.appendContent("more stuff")
		content: string	
resetContent()	N/A	Resets the body of the response to empty.	response.resetContent()
end()	N/A	Initiates network transfer for the response.	response.setContent("Hello, world!") response.end() response.setPath("my-file.txt") response.end()
setContent(content)	N/A	Sets the body of the response to <i>content</i> .	response.setContent("Hello, world!")
		content: string	
setToFile(path)	N/A	Sets content of the response to the contents of the file at <i>path</i> . The path must be local to the device running the server.	response.setToFile("intro.html")
		path: string	
setToNotFound()	N/A	Canned response for when a requested resource can not be located.	response.setToNotFound()
send(content)	N/A	Sends <i>content</i> as plain text.	response.send("Hello, world!")
		content: string	
setContentType(type)	N/A	Sets <i>Content-Type</i> header to <i>type</i> .	response.setContentType("text/html")
		type: string	
sendFile(path)	N/A	Sends the contents of the file at <i>path</i> .	response.send("my-file.txt")
		path: file path local to the device running the server.	
sendNotFound()	N/A	Sends canned response for when a requested resource can not be located.	response.sendNotFound()

Real HTTP Client (External Network Access)

Package = realhttp

Function	Return Type	Description	Example
RealHTTPClient()	RealHTTPClient	Creates a Real HTTP Client.	http = RealHTTPClient()
get(url)	N/A	Gets an URL.	http.get("http://www.cisco.com")
post(url, data)	N/A	Posts data to an URL. data can be a string or a dictionary; if dictionary, it will be URL-encoded into the body.	http.post(url, {"num":1, "str":"hello"})
put(url, data)	N/A	Puts data to an URL. data can be a string or a	http.put(url, {"num":1, "str":"hello"})

dictionary; if dictionary, it will be URL-encoded into the body.

deleteResource(url)	N/A	Sends a delete to an URL.	http.deleteResource(url)
getWithHeader(url, header)	N/A	Gets an URL with custom header fields as a dictionary.	http.getWithHeader("https://api.ciscospark.com/v1/people/me", {"Authorization": "Bearer xxyyzz"})
postWithHeader(url, data, header)	N/A	Posts data to an URL with custom header fields as a dictionary. data can be a string or a dictionary; if dictionary and custom header field has "application/json" as the "content-type", it will be json-encoded, otherwise it will be URL-encoded into the body.	http.postWithHeader("https://api.ciscospark.com/v1/messages", {"toPersonId": "722bb271-d7ca-4bce-a9e3-471e4412fa77", "text": "Hi Sparky"}, {"Authorization": "Bearer xxyyzz", "Content-Type": "application/json"})
putWithHeader(url, data, header)	N/A	Puts data to an URL with custom header fields as a dictionary. data can be a string or a dictionary; if dictionary and custom header field has "application/json" as the "content-type", it will be json-encoded, otherwise it will be URL-encoded into the body.	http.putWithHeader("https://api.ciscospark.com/v1/rooms/xxyyzz", {"title": "New room name"}, {"Authorization": "Bearer xxyyzz"})
deleteResourceWithHeader(url, header)	N/A	Sends a delete to an URL with custom header fields as a dictionary.	http.deleteResourceWithHeader("https://api.ciscospark.com/v1/messages/xxyyzz", {"Authorization": "Bearer xxyyzz"})
onDone(callback)	N/A	Sets the callback for when the request is done. replyHeader is a dictionary of header fields in the reply. It is added to 7.1 and is optional.	def onHTTPOne(status, data, replyHeader): print(data) ... http.onDone(onHTTPOne)

Real WebSocket Client (External Network Access)

Package = realhttp

Function	Return Type	Description	Example
RealWSCient()	RealWSCient	Creates a Real WebSocket Client.	client = RealWSCient()
connect(url)	N/A	Connects to a server URL	client .connect("ws://1.1.1.1:2000")
connected()	bool	Returns true if connected.	client.connected()
close ()	N/A	Disconnects.	client . close ()

state ()	int	Returns the state of the connection.	client.state()
remoteIP ()	str	Returns the remote IP.	client .remoteIP()
remoteHost()	str	Returns the remote server host name.	client.remoteHost()
remotePort()	int	Returns the remote port.	client .remotePort()
localIP()	str	Returns the local IP.	client.localIP()
localPort()	int	Returns the local port.	client.localPort()
send(data)	N/A	Sends data to remote connection.	client .send("hello")
error()	int	Returns the last error code.	client.error()
errorString()	str	Returns the last error in string.	client.errorString()
onReceive(callback)	N/A	Sets the callback for when data is received.	def onWSReceive(data): print(data) ... client.onReceive(onWSReceive)
onConnectionChange(callback)	N/A	Sets the callback for when the connection changes.	def onWSConnectionChange(type): print(type) ... client.onConnectionChange(onWSConnectionChange)

Real TCP (External Network Access)

Package = realtcp

Function	Return Type	Description	Example
RealTCPClient()	RealTCPClient	Creates a Real TCP Client.	client = RealTCPClient()
connect(ip, port)	N/A	Connects to ip and port.	client.connect("1.1.1.1", 2000)
connected()	bool	Returns true if connected.	client.connected()
close()	N/A	Disconnects.	client.close()
state()	int	Returns the state of the connection.	client.state()
remoteIP()	str	Returns the remote IP.	client.remoteIP()
remoteHost()	str	Returns the remote server host name.	client.remoteHost()
remotePort()	int	Returns the remote port.	client.remotePort()
localIP()	str	Returns the local IP.	client.localIP()
localPort()	int	Returns the local port.	client.localPort()
send(data)	N/A	Sends data to remote connection.	client.send("hello")
error()	int	Returns the last error code.	client.error()
errorString()	str	Returns the last error in string.	client.errorString()
onReceive(callback)	N/A	Sets the callback for when data is received.	def onTCPReceive(data): print(data) ... client.onReceive(onTCPReceive)
onConnectionChange(callback)	N/A	Sets the callback for when the connection changes.	def onTCPConnectionChange(type): print(type) ... client.onConnectionChange(onTCPConnectionChange)
RealTCPServer()	RealTCPServer	Creates a Real TCP Server.	server = RealTCPServer()
listen(port)	bool	Listens on port on host device and returns whether it was successful.	server.listen(1234)
isListening()	bool	Returns true if listening.	server.isListening()
stop ()	N/A	Stops listening. All connections remain open.	server.stop()
listeningIP()	str	Returns the listening IP.	server.listeningIP()
listeningPort()	int	Returns the listening port.	server.listeningPort()
error()	int	Returns the last error code.	server.error()
errorString()	str	Returns the last error in string.	server.errorString()

onNewClient(callback)	N/A	Sets the callback for when a new client is connected.	<pre>def onTCPNewClient(client): print("new client") ... server.onNewClient(onTCPNewClient)</pre>
-----------------------	-----	---	--

Real UDP (External Network Access)

Package = realudp

Function	Return Type	Description	Example
RealUDPSocket()	RealUDPSocket	Creates an Real UDP Socket.	udp = RealUDPSocket()
begin(port)	bool	Starts listening on port.	udp.begin(2000)
stop()	N/A	Stops listening.	udp.stop()
joinMulticastGroup(ip)	bool	Joins a multicast group. Must call begin() first.	udp.joinMulticastGroup("224.0.0.1")
leaveMulticastGroup(ip)	bool	Leaves a multicast group.	udp.leaveMulticastGroup("224.0.0.1")
localIP()	str	Returns the local IP.	udp.localIP()
localPort()	int	Returns the local port.	udp.localPort()
send(ip, port, data)	bool	Sends data.	udp.send("1.1.1.1", 2000, "hello")
onReceive(callback)	N/A	Sets the callback for when data is received.	<pre>def onUDPReceive(ip, port, data): print(data) ... udp.onReceive(onUDPReceive)</pre>
error()	int	Returns the last error code.	udp.error()
errorString()	str	Returns the last error in string.	udp.errorString()

JSON

Package = json

Function	Return Type	Description	Example
dumps(obj)	str	Serializes a Python object into JSON string.	jsonStr = json.dumps({"num":1, "s":"str"})
loads(jsonStr)	python object	Converts a JSON string into a Python object.	obj = json.loads(s)

GUI (SBC and Thing , some End Devices)

Package = gui

Function	Return Type	Description	Example
GUI.setup()	N/A	Initializes the GUI and tells the GUI to display the html file configured in the manifest file. If the app is not installed, it throws an error.	GUI.setup()
GUI.update(type, args)	N/A	Asynchronously calls the html file's update() function passing in type and args. Type is a string and args can be any JSON valid object.	GUI.update("status", "playing")
guiEvent(type, args)	N/A	If defined, this function is called asynchronously when the html file calls guiEvent(type, args). Type is a string and args can	<pre>def guiEvent(type, args): print('guiEvent: ' + type + ',' + dumps(args));</pre>

be any JSON valid object.

CLI (SBC and Thing, some End Devices)

Package = cli

Function	Return Type	Description	Example
CLI.setup()	N/A	Initializes the CLI and will call the cliEvent() function if the app was started from a command in the Command Prompt.	CLI.setup()
CLI.exit()	N/A	Exits the CLI mode and returns back to the Command Prompt.	CLI.exit()
cliEvent(type, args)	N/A	If defined, this function is called synchronously when different CLI events happen. Type is a string and args can be any JSON valid object. Type can be: "invoked" - this is called when the command is invoked from the Command Prompt; args is a list of command arguments where the first element is the command name. "interrupted" - this is called when the user uses break shortcut (Ctrl+C) while this app is running in CLI mode.	<pre>def cliEvent(type, args): print('cliEvent: ' + type + ',' + dumps(args));</pre>

Simulation

Package = simulation

Function	Return Type	Description	Example
Simulation.isInSimulationMode()	bool	Returns whether PT is currently in Simulation Mode.	if Simulation.isInSimulationMode(): print("In sim mode")
Simulation.addCustomProtocolType(protocol)	bool	Adds a new protocol type to the current file/network. The protocol will show up in the Event List Filters. Returns true if successful, false otherwise.	Simulation.addCustomProtocolType("MyProtocol")
Simulation.hasCustomProtocolType(protocol)	bool	Returns whether the protocol type is already added to the current file/network.	if not Simulation.hasCustomProtocolType("MyProtocol"): Srr

`Simulation.addCustomPDU(protocol, pduType, definition)`

bool

Adds a new protocol type and PDU type to the current file/network. The protocol will show up in the Event List Filters, and the PDU with its definition will show up in PDU details. Returns true if successful, false if the definition is invalid. It will replace existing definitions if using the same protocol and PDU type names.

The definition argument is an object with the following properties describing the layout and fields of the PDU:

- title - a string that shows up as the title in PDU Details
- units - a string
- unit_marks - an array of numbers indicating where the marks are to show up
- width - an integer indicating the width of PDU
- fields - an array of objects representing each field in the PDU; each object must contain the following properties:
 - value - a label that can contain a variable inside curly braces; the variable will be replaced with a value in the PDUInfo fields object
 - size - an integer indicating the size of the field

`Simulation.addCustomPDU("MyProtocol", "MyPDU"`

`Simulation.hasCustomPDU(protocol, pduType)`

bool

Returns whether the protocol and PDU type is already added to the current file/network.

if not `Simulation.hasCustomPDU("MyProtocol", "MyPDU")`: #

`PDUInfo(color)`

PDUInfo

Creates a PDU Info object with the pdu color. A PDUInfo is required to show PDU with more details in Simulation Mode.

`pduInfo = PDUInfo(0xffff00)`

`setOutFormat(protocol, pduType, fields)`

N/A

Sets the outgoing PDU to be displayed in a custom PDU definition. The fields argument is an object with the variables defined in the PDU definition fields property.

`pduInfo.setOutFormat("MyProtocol", "MyPDU", {"type": "REF`

`addOutMessage(message)`

N/A

Adds a message to the outgoing OSI layer 7.

`pduInfo.addOutMessage("I am sending some data.")`

`addInMessage(message)`

N/A

Adds a message to the incoming OSI layer 7.

`pduInfo.addInMessage("I received some data.")`

setAccepted()	N/A	Sets the PDU as accepted.	pduInfo.setAccepted()
setDropped()	N/A	Sets the PDU as dropped.	pduInfo.setDropped()

Workspace**Package = workspace**

Function	Return Type	Description	Example
getPhysicalObject()	PhysicalObject	Returns the current device's physical object.	po = getPhysicalObject()
getName()	str	Returns the name of this physical object.	print(po.getName())
getType()	int	Returns the type of this physical object. Valid values are: INTER_CITY = 0CITY = 1BUILDING = 2WIRING_CLOSET = 3RACK = 4TABLE = 5DEVICE = 6MULTIUSER = 7GENERIC_CONTAINER = 8	print(po.getType())
getX()	float	Returns the x coordinate relative to its parent in meters.	print(po.getX())
getY()	float	Returns the y coordinate relative to its parent in meters.	print(po.getY())
getCenterX()	float	Returns the center x coordinate relative to its parent in meters.	print(po.getCenterX())
getCenterY()	float	Returns the center y coordinate relative to its parent in meters.	print(po.getCenterY())
getGlobalX()	float	Returns the global x coordinate of this physical object in meters.	print(po.getGlobalX())
getGlobalY()	float	Returns the global y coordinate of this physical object in meters.	print(po.getGlobalY())
getWidth()	float	Returns the width of this physical object in meters.	print(po.getWidth())
getHeight()	float	Returns the height of this physical object in meters.	print(po.getHeight())
moveTo(x, y)	N/A	Moves this physical object's top left corner to the specified coordinates in meters relative to its parent.	po.moveTo(10, 20)
moveCenterTo(x, y)	N/A	Moves this physical object's center to the specified coordinates in meters relative to its parent.	po.moveCenterTo(10, 20)
moveBy(x, y)	N/A	Moves this physical object by the specified coordinates in meters.	po.moveBy(10, 20)
setVelocity(x, y)	N/A	Sets this physical object's velocity in meters. This physical object then moves automatically.	po.setVelocity(1, 2)
getXVelocity()	float	Returns the x velocity of this physical object in meters.	print(po.getXVelocity())
getYVelocity()	float	Returns the y velocity of this physical object in meters.	print(po.getYVelocity())
getParent()	PhysicalObject	Returns this physical object's parent.	parentPO = po.getParent()
moveOutOfParent()	bool	Moves this physical object out of its parent to the same level as the parent, and returns whether it was successful.	po.moveOutOfParent()
moveInto(name)	bool	Moves this physical object into a container with the specified name that is in the same level as this physical object, and returns whether it was successful.	po.moveInto("Corporate Office")
getChildCount()	int	Returns the number of children this physical object has.	print(po.getChildCount())
getChildAt(index)	PhysicalObject	Returns the child physical object at the specified index.	childPO = po.getChildAt(0)
getChild(name)	PhysicalObject	Returns the child physical object with the specified name.	childPO = po.getChild("Wiring Closet")
getLogicalObject()	LogicalObject	Returns the current device's logical object.	lo = getLogicalObject()
getName()	str	Returns the name of this logical object.	print(lo.getName())
getType()	int	Returns the type of this logical object. Valid values are: ROOT = 1099DEVICE = 1100CLUSTER = 1104MULTIUSERITEM = 1108	print(lo.getType())
getX()	int	Returns the x coordinate relative to its parent in pixels.	print(lo.getX())
getY()	int	Returns the y coordinate relative to its parent in pixels.	print(lo.getY())
getCenterX()	int	Returns the center x coordinate relative to its parent in pixels.	print(lo.getCenterX())
getCenterY()	int	Returns the center y coordinate relative to its parent in pixels.	print(lo.getCenterY())
getWidth()	int	Returns the width of this logical object in pixels.	print(lo.getWidth())

getHeight()	int	Returns the height of this logical object in pixels.	print(lo.getHeight())
moveTo(x, y)	N/A	Moves this logical object's top left corner to the specified coordinates in pixels relative to its parent.	lo.moveTo(10, 20)
moveCenterTo(x, y)	N/A	Moves this logical object's center to the specified coordinates in pixels relative to its parent.	lo.moveCenterTo(10, 20)
moveBy(x, y)	N/A	Moves this logical object by the specified coordinates in pixels .	lo.moveBy(10, 20)
setVelocity(x, y)	N/A	Sets this logical object's velocity in pixels . This logical object then moves automatically.	lo.setVelocity(1, 2)
getXVelocity()	float	Returns the x velocity of this logical object in pixels.	print(lo.getXVelocity())
getYVelocity()	float	Returns the y velocity of this logical object in pixels.	print(lo.getYVelocity())
getParent()	LogicalObject	Returns this logical object's parent.	parentLO = lo.getParent()
moveOutOfParent()	bool	Moves this logical object out of its parent to the same level as the parent, and returns whether it was successful.	lo.moveOutOfParent()
moveInto(name)	bool	Moves this logical object into a container with the specified name that is in the same level as this logical object, and returns whether it was successful.	lo.moveInto("Cluster0")
getChildCount()	int	Returns the number of children this logical object has.	print(lo.getChildCount())
getChildAt(index)	LogicalObject	Returns the child logical object at the specified index.	childlo = lo.getChildAt(0)
getChild(name)	LogicalObject	Returns the child logical object with the specified name.	childlo = lo.getChild("Cluster0")

Bluetooth Manager**Package = bluetooth**

Function	Return Type	Description	Example
init()	N/A	Initialize Bluetooth functionality for the device.	def main(): Bluetooth.init()
onDeviceDiscovered(macAddress, deviceName)	N/A	This event handler is called when a device is discovered.	def handleDeviceDiscovered(macAddress, deviceName):... Bluetooth.onDeviceDiscovered(handleDeviceDiscovered) Bluetooth.discover()
onDevicePaired(macAddress)	N/A	This event handler is called when a device is paired.	def handleDevicePaired(macAddress):... Bluetooth.onDevicePaired(handleDevicePaired)
onDeviceUnPaired(macAddress)	N/A	This event handler is called when a device is unpaired.	def handleDeviceUnpaired(macAddress):... Bluetooth.onDeviceUnpaired(handleDeviceUnpaired)
onDeviceConnected(macAddress)	N/A	This event handler is called when a device is connected.	def handleDeviceConnected(macAddress):... Bluetooth.onDeviceConnected(handleDeviceConnected)
onDeviceDisconnected(macAddress)	N/A	This event handler is called when a device is disconnected.	def handleDeviceDisconnected(macAddress):... Bluetooth.onDeviceDisconnected(handleDeviceDisconnected)
setDiscoverable(discoverable)	N/A	Sets the Bluetooth device to discoverable or not.	Bluetooth.setDiscoverable(true)
isDiscoverable()	bool	Returns true if discoverable otherwise false.	Bluetooth.isDiscoverable()
discover()	N/A	Starts the discovering process. If devices are discovered, the onDeviceDiscovered() event will trigger.	Bluetooth.discover()
pair(mac)	N/A	Starts pairing with mac address.	Bluetooth.pair(macAddress)
unpair(mac)	N/A	Starts unpairing with mac address.	Bluetooth.unpair(macAddress)
onPairRequest(macAddress, deviceName)	N/A	This event handler is called if there's a pair request and the Bluetooth manager is set to accept pair requests.	Bluetooth.setAcceptingPairRequest(true) def handlePairRequest(macAddress, deviceName):... Bluetooth.onPairRequest = handlePairRequest
setAcceptingPairRequest(accepting)	N/A	Sets the Bluetooth manager to accept pair requests	Bluetooth.setAcceptingPairRequest(true)

acceptPairRequest(macAddress, deviceName)	N/A	Accepts the pair request with macAddress and deviceName	Bluetooth.acceptPairRequest(macAddress, deviceName)
getPairedDevices()	Array	Returns an array of paired Bluetooth devices.	var devices = Bluetooth.getPairedDevices()
getConnectedDevices()	Array	Returns an array of connected devices.	var devices = Bluetooth.getConnectedDevices()
enableBroadcast()	N/A	Enables Bluetooth broadcast.	Bluetooth.enableBroadcast()
isBroadcastEnabled()	bool	Returns true if broadcast is enabled.	Bluetooth.isBroadcastEnabled()
broadcastBeacon(beaconUuid, data)	bool	Broadcast beacon with beaconUuid and data.	Bluetooth.broadcastBeacon(uuid, data)

Bluetooth Service**Package = bluetooth**

Function	Return Type	Description	Example
start(uuid)	N/A	Starts the Bluetooth service.	<pre>bts = BluetoothService() dstService = "{00000000-0000-0000-0000-000000000001}" bts.start(dstService)</pre>
connect(dstMac, dstServiceUuid)	bool	Returns true if connected to dstMac and dstServiceUuid; false otherwise.	bts.connect(dstMac, dstServiceUuid)
stop()	N/A	Stops the Bluetooth service.	bts.stop()
send(dstMac, dstServiceUuid, data)	N/A	Sends data to dstMac and dstServiceUuid with data.	bts.send(dstMac, dstServiceUuid, "hello")
sendToConnected(data)	N/A	Sends data to connected device.	bts.sendToConnected("hello")
onReceive(srcMac, srcService, dstMac, dstService, data)	N/A	This event handler receives data from srcMac, srcService, dstMac and dstService with data.	<pre>def handleReceive(srcmac, srcData, dstMac, dstService, data){...} bts.onReceive(handleReceive)</pre>

Requests Module**Package = requests**

Function	Return Type	Description	Example
get(url, params=None, **options)	Response	Make a GET request. Supported options: timeout, headers, params. <i>params</i> parameter/option can be in the following forms: 1. {key: value, key: value, ...} 2. [(key, value), (key, value), ...] 3. ((key, value), (key, value), ...) 4. key=value&key=value&... # NOTE! this is a string, where keys and values must be URL-encoded	reply = requests.get(url)
post(url, data=None, json=None, **options)	Response	Make a POST request. Supported options: timeout, headers, json, data. <i>data</i> parameter/option Examples: 1. any structure suitable for params for get above. In this case it will be sent in the body 2. a string - will be sent AS IS <i>json</i> parameter/option Examples: 1. a valid(!) JSON string 2. any Python structure that <i>json</i> library <i>dumps()</i> can work with	<pre>reply = requests.post(url, json={ "username":"admin", "password":"hello"}) reply = requests.post(url, data= ("username","admin"), ("password","hello")) reply = requests.post(url, data="whatever data needs sending as a string")</pre>
put(url, data=None, **options)	Response	Make a PUT request. Supported options: timeout, headers, data. <i>data</i> parameter/option - see post method above.	reply = requests.put(url, data="whatever data needs to be stored as a string", headers=..., timeout=10)
delete(url, **options)	Response	Make a DELETE request. Supported options: timeout, headers. <i>data</i> parameter/option - see post method above.	reply = requests.delete(url, headers=...)

request(method, url, **options)	Response	Make an HTTP request using a given <i>method</i> . Supported options: timeout, headers, json, data, params. Depending on the method used. <i>data</i> parameter/option - see post method above.	reply = requests.request("POST", url, data=..., headers=...)
urlencode(data)	string	URL-encodes <i>data</i> in a way suitable for using with GET parameters Returns input data encoded.	reply = requests.urlencode("hello world") is expected to return "hello%20world"
Response	class	Properties: <i>request</i>: original <i>Request</i> object. <i>status_code</i>: integer HTTP status code (200 is OK, 404 is Not Found, etc). <i>data</i> : data returned as a string. NOTE: this value is NOT in bytes. <i>ok</i> : True for success, False - otherwise. <i>text</i> : same as <i>data</i>. <i>content</i> : same as <i>data</i>. <i>url</i> : original request url. Methods: <i>json()</i> : Returns a Python object representing JSON result when applicable.	
Request	class	Properties: <i>url</i>: HTTP request url. <i>method</i>: string - one of GET, DELETE, POST, PUT. <i>headers</i> : dictionary containing request headers. <i>timeout</i> : request timeout in seconds. Default value of 0 means - no timeout. <i>data</i> : data as per API specification. <i>json</i> : JSON data as per API specification. <i>params</i> : a dictionary containing request URL parameters.	

Python in Cisco Packet Tracer

This section contains notes on use of Python in scripting Cisco Packet Tracer devices. Here we refer to Cisco Packet Tracer Python as PT-Python and use C-Python to refer to the standalone Python proper.

- PT-Python interpreter comes from an open source project *Skulpt*. At the time of this writing that project's main page can be found [here](#). Please refer to the documentation available on the main project's site above. PT-Python uses version of *Skulpt* that is roughly compatible with C-Python 2.7(ish). Noteworthy, PT-Python version used by any given installation of Cisco Packet Tracer is likely to be older than the most-recent one documented on the pages of *Skulpt* project. Even more relevant information can be found at the *Skulpt* github project the user is encouraged to seek out.
- In PT-Python it is not possible to use modules generally available for C-Python. This also precludes the use of C-Python package managers like *pip* or similar.
- C-Python builtin APIs dedicated for accessing host computer environment, such as processes, file system, network, etc., cannot be used in PT-Python scripts.
- Given PT-Python interpreter is designed to run inside web pages and due to the nature of its Javascript-based runtime, script execution speed is reduced when compared to C-Python.